

Proj1: 数据的捕获与构建

1 Step1: 领域选择与目标定义

1.1 领域选择

选定领域: 电商交易

领域说明: 选取真实的电商平台交易数据。该领域数据维度丰富, 包含时间戳、地理位置、价格、评分等多种数据类型。

关键实体集: 至少包含 4 个核心实体, 符合复杂性要求:

1. **Customer (客户)**: 包含客户ID、所在城市、州等。
2. **Product (商品)**: 包含商品ID、类别、尺寸、重量等。
3. **Order (订单)**: 包含订单ID、购买时间、物流状态、预计送达时间等。
4. **Seller (卖家)**: 包含卖家ID、所在地等。

1.2 数据源调研

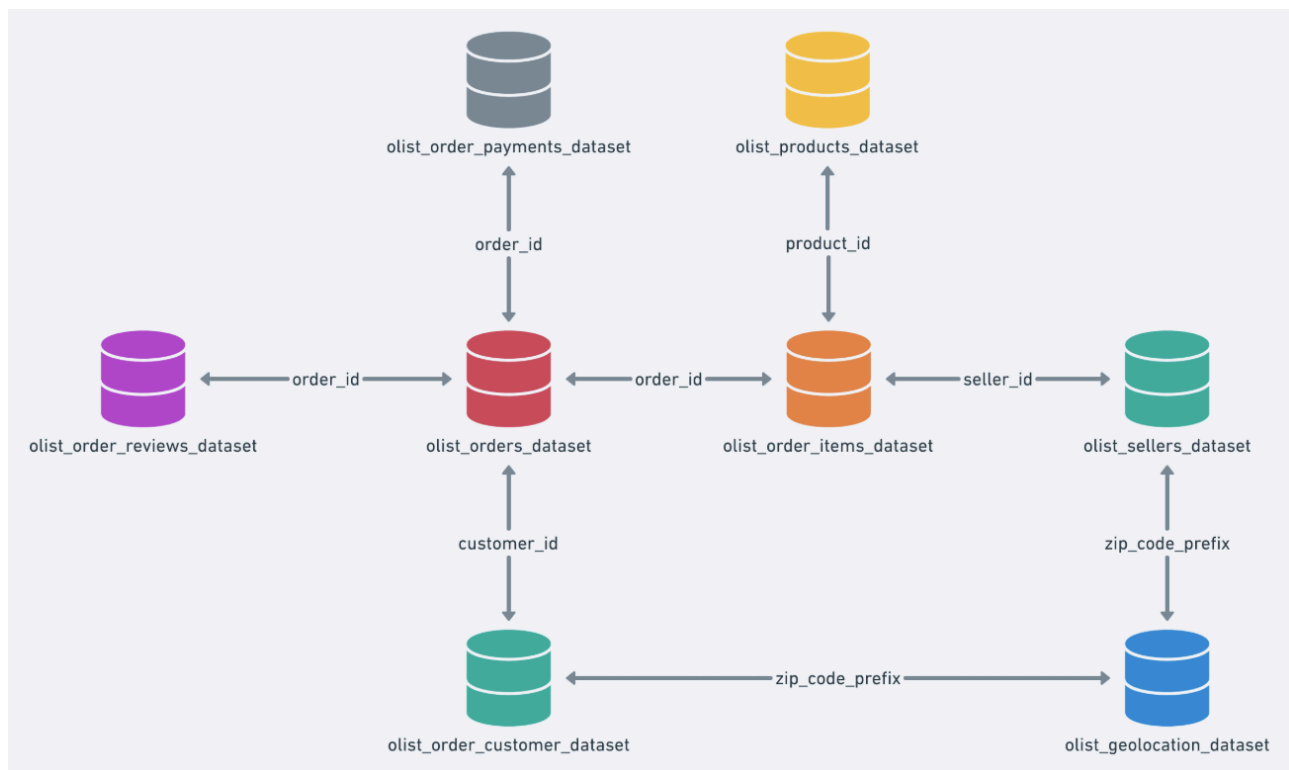
数据来源: Kaggle开源数据集 - Brazilian E-Commerce Public Dataset by Olist

数据源链接: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

数据规模与维度:

- **规模:** 包含超过 90,000 条真实的匿名交易记录。
- **维度:** 数据以多张 CSV 表格形式提供, 天然需要进行外键关联。包含数值型 (价格、运费、长宽高)、日期型 (下单时间、发货时间、送达时间) 以及文本型 (商品类目、用户评价)。

数据源文件的关系如下:



1.3 提出分析目标

1. 物流时效与客户满意度关联分析：分析 2017-2018 年已送达订单中，实际送达时间晚于预计送达时间的订单，其订单级平均评分是否比按时送达订单低 1 分以上？
 - **【动词】**：分析
 - **【具体指标】**：订单级平均评分
 - **【比较/条件】**：实际送达时间晚于预计送达时间的订单，对比按时送达订单，评分差异是否超过 1 分
 - **【时间范围/细分领域】**：2017-2018 年已送达电商订单
2. 品类消费与复购行为差异分析：对比 2017-2018 年购买过家居家装类商品的客户与购买过美妆个护类商品的客户，其平均订单总金额与复购率是否存在明显差异？
 - **【动词】**：对比
 - **【具体指标】**：平均订单总金额、复购率
 - **【比较/条件】**：家居家装类客户对比美妆个护类客户
 - **【时间范围/细分领域】**：2017-2018 年电商客户消费行为
 - **【补充】**：在数据中，家居家装类包括 `moveis_decoracao`、`moveis_quarto`、`moveis_sala`、`cama_mesa_banho`、`utilidades_domesticas`、`eletrodomesticos`；美妆个护类包括 `beleza_saude`、`perfumaria`。复购率定义为统计期内下单数大于等于 2 的客户占比
3. 季度运费成本趋势分析：侦测 2017 年已送达订单中，第四季度订单的平均运费占订单商品总价比例是否比前三个季度更高。
 - **【动词】**：侦测
 - **【具体指标】**：平均运费占订单商品总价比例
 - **【比较/条件】**：第四季度订单对比前三个季度订单
 - **【时间范围/细分领域】**：2017 年已送达电商订单

2 Step2: 数据获取与初始入库

2.1 数据获取

数据来源： Kaggle 开源数据集 Brazilian E-Commerce Public Dataset by Olist。原始数据以 8 张 CSV 表格形式提供，本项目使用其中 5 张：

文件名	说明	主要字段
olist_customers_dataset.csv	客户表	customer_id, customer_unique_id
olist_orders_dataset.csv	订单表	order_id, customer_id, 购买/送达/预计送达时间戳
olist_order_items_dataset.csv	订单明细表	order_id, order_item_id, product_id, price, freight_value
olist_products_dataset.csv	商品表	product_id, product_category_name
olist_order_reviews_dataset.csv	评价表	review_id, order_id, review_score

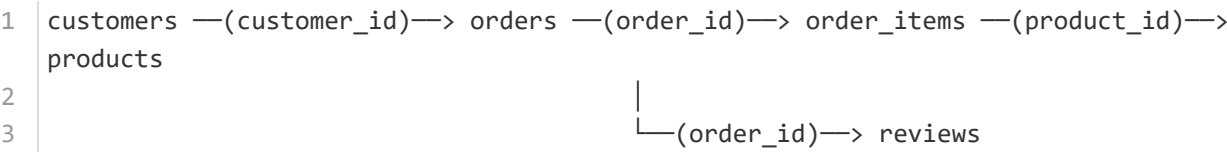
数据规模： 原始订单记录约 99,441 条，订单明细约 112,000 条，商品约 32,000 种，客户约 96,000 位。

2.2 数据清洗

清洗工作由 Python 脚本 `clean_flat_table.py` 完成，使用 `pandas` 库进行数据处理。脚本的核心流程如下：

1. 数据合并流程

脚本从 `archive/` 目录读取上述 5 个 CSV 文件，仅选取分析所需的列，然后通过 `inner join` 逐步合并：



合并后得到一张包含所有维度的大宽表，每行代表一个订单明细与评价的组合。

2. 缺失值处理策略

字段	处理方式	理由
order_delivered_customer_date	丢弃该行（删除未送达订单）	三个分析目标均依赖已送达订单的时间数据，未送达订单无法参与物流时效分析
product_category_name	填充为 'Unknown'	保留交易记录完整性，避免因类别缺失而丢失价格、运费等有效数据
review_score	填充为 3（中位值）	评分 1-5 分，3 为中间值，减少对均值的极端拉偏

3. 脏数据处理

三个时间戳字段（`order_purchase_timestamp`、`order_delivered_customer_date`、`order_estimated_delivery_date`）通过 `pd.to_datetime(errors='coerce')` 统一解析，输出时格式化为 `%Y-%m-%d %H:%M:%S`，确保时间格式一致。

4. 清洗结果统计

指标	数值
原始订单数	99,441
丢弃未送达订单数	2,965
清洗后订单数	96,476
商品类别原始缺失数	610
最终宽表行数	110,012
最终宽表列数	12

最终宽表 `cleaned_flat_table.csv` 包含以下 12 列：`order_id`、`order_item_id`、`customer_unique_id`、`order_purchase_timestamp`、`order_delivered_customer_date`、`order_estimated_delivery_date`、`product_id`、`product_category_name`、`price`、`freight_value`、`review_id`、`review_score`。

行数（110,012）大于订单明细数，原因是部分订单存在多条评价记录，合并后保留了每条评价与对应订单明细的组合。每行通过 `(order_id, order_item_id, review_id)` 唯一标识。

5. 运行方式

```
1 | python clean_flat_table.py --input-dir archive --output cleaned_flat_table.csv
```

2.3 初始数据库实现

2.3.1 openGauss 部署

采用 Docker Desktop 部署 openGauss 官方镜像（`opengauss/opengauss:latest`），从 Docker Hub 直接拉取。容器启动命令如下：

```
1 | docker run \
2 |   --env=GS_PASSWORD=MyGauss@123 \
3 |   --volume=E:\Docker\OpenGauss:/var/lib/opengauss \
4 |   -p 15432:5432 \
5 |   -d opengauss/opengauss:latest
```

关键配置说明：

- 端口映射：宿主机 `15432` → 容器内 `5432`，外部通过 `localhost:15432` 连接。
- 数据持久化：通过 `-v` 将宿主机 `E:\Docker\OpenGauss` 挂载到容器内 `/var/lib/opengauss`，数据库文件持久化存储。
- 数据库超级用户：`omm`（openGauss 默认），通过 `GS_PASSWORD` 环境变量设置初始密码。

2.3.2 初始表结构设计

本阶段采用非规范化的大宽表方案，将清洗后的所有数据存入单张表中，对应 DDL 脚本为

`schema.sql`：

```
1 | CREATE TABLE cleaned_flat_table (
2 |     id SERIAL PRIMARY KEY,
3 |     order_id VARCHAR(32) NOT NULL,
4 |     order_item_id INTEGER NOT NULL CHECK (order_item_id >
5 | 0),
6 |     customer_unique_id VARCHAR(32) NOT NULL,
```

```

6      order_purchase_timestamp    TIMESTAMP    NOT NULL,
7      order_delivered_customer_date  TIMESTAMP    NOT NULL,
8      order_estimated_delivery_date  TIMESTAMP    NOT NULL,
9      product_id                  VARCHAR(32)   NOT NULL,
10     product_category_name        VARCHAR(100)  NOT NULL DEFAULT 'Unknown',
11     price                        NUMERIC(10,2)  NOT NULL CHECK (price >= 0),
12     freight_value                NUMERIC(10,2) NOT NULL CHECK (freight_value >=
0),
13     review_id                    VARCHAR(32)   NOT NULL,
14     review_score                  INTEGER       NOT NULL DEFAULT 3
15                                     CHECK (review_score BETWEEN 1 AND 5),
16     UNIQUE (order_id, order_item_id, review_id)
17 );

```

完整性约束设计说明：

- 主键： `id SERIAL`，自增整数代理主键，索引高效，适合过渡性宽表。
- UNIQUE： `(order_id, order_item_id, review_id)` 组合唯一约束，保证记录无重复。一个订单可包含多个商品项，一个订单可有多条评价，三者联合唯一标识每条记录。
- NOT NULL：所有列均设为 NOT NULL，因清洗脚本已处理所有缺失值。
- DEFAULT： `product_category_name` 默认 'Unknown'， `review_score` 默认 3，与清洗逻辑保持一致。
- CHECK： `price >= 0` 和 `freight_value >= 0`（金额不为负）； `order_item_id > 0`（项号从 1 开始）； `review_score BETWEEN 1 AND 5`（评分范围限制）。

2.3.3 导入 Navicat

id # int4	order_id # varchar(32)	order_item_id # int4	customer_unique_id # varchar(32)	order_purchase_time # timestamp(6)	order_delivered_cust # timestamp(6)	order_estimated_deli # timestamp(6)	product_id # varchar(32)	product_category_na # varchar(100)	price # numeri	review_id # varchar(32)	freight_value # numeric(10,2)	review_score # int4
1	e481f51cbd	1	7c396fd4830fd04220f75	2017-10-02 10:56:33	2017-10-10 21:25:13	2017-10-18 00:00:00	87285b348845	utilidades_domesticas	29.99	a54f0611ad9ed256b57	8.72	4
2	53cdbc2fcb	1	a0f7308b275d755c9edb	2018-07-24 20:41:37	2018-08-07 15:27:45	2018-08-13 00:00:00	595fac2a385ac	perfumaria	118.70	8d5266042046a06655c	22.76	4
3	47770eb91c	1	3ae53a41f69f3d2a113	2018-08-08 08:38:49	2018-08-17 18:06:29	2018-09-04 00:00:00	aa4383b373c6	automotivo	159.90	e73b67b67587f7644d5t	19.22	5
4	949d5b44dl	1	7c142cf63193a1473d2e	2017-11-18 19:28:06	2017-12-02 00:28:42	2017-12-15 00:00:00	d0b61bf1de8b	pet_shop	45.00	359d036e76b3c069f62c	27.20	5
5	ad21c59c08	1	72632f0f9dd73dfce990c	2018-02-13 21:18:39	2018-02-16 18:17:02	2018-02-26 00:00:00	65266b2da20d	papelaria	19.90	e50934922425744ba8c	8.72	5
6	a4591c265e	1	80bb27c7c16e8973207	2017-07-09 21:57:05	2017-07-26 10:57:55	2017-08-01 00:00:00	060cb19345d9	automotivo	147.90	89b738e70a1ce346db2t	27.36	4
7	6514b8ad8c	1	932afa1e70822e5821d	2017-05-16 13:10:30	2017-05-26 12:55:51	2017-06-07 00:00:00	4520766ec412	automotivo	59.99	07d67dd06ed5f88b8ef1	15.17	5
8	76c6e86628	1	39382392765bdc7481c	2017-01-23 18:29:09	2017-02-02 14:08:10	2017-03-06 00:00:00	ac1789e492dc	moveis_decoracao	19.90	fc4af8aebec3f1a3cd18t	16.05	1
9	e69bf5eb8b	1	299905e3934e9e181bf	2017-07-29 11:55:02	2017-08-16 17:14:30	2017-08-23 00:00:00	9a78fb9862b1t	moveis_escritorio	149.99	abc5655186d40772bd6f	19.77	5
10	e6ce16cb79	1	f2a85dec752b8517b5e5	2017-05-16 19:41:10	2017-05-29 11:18:31	2017-06-07 00:00:00	08574b074924	ferramentas_jardim	99.00	15898b543726a832d41c	30.53	1
11	e6ce16cb79	2	f2a85dec752b8517b5e5	2017-05-16 19:41:10	2017-05-29 11:18:31	2017-06-07 00:00:00	08574b074924	ferramentas_jardim	99.00	15898b543726a832d41c	30.53	1
12	34513ce0c4	1	782987b81c92239d922	2017-07-13 19:58:11	2017-07-19 14:04:48	2017-08-08 00:00:00	f7e0fa615b386	informatica_acessorios	98.00	a932e1fddc0c17fc26de	16.13	4
13	82566ae60a	1	e97109680b052ee858d	2018-06-07 10:06:19	2018-06-19 12:05:52	2018-07-18 00:00:00	72a97c271b2e	perfumaria	31.90	ed220e853e96b629118	18.23	5
14	5ff96c15d0t	1	e2dfda3127fedbbca9707f	2018-07-25 17:44:10	2018-07-30 15:52:25	2018-08-08 00:00:00	10adb53d8faa	cama_mesa_banho	19.90	346e21b1b57a959594c	12.80	5
15	432aa21d8f	1	04cf8185c71090d28bba	2018-03-01 14:14:28	2018-03-12 23:36:26	2018-03-21 00:00:00	72d3bf1d3a79f	brinquedos	38.25	8f39f09c22b1ceb6e3eb	16.11	4
16	dc3b6b511f	1	ccaf1c3f270410521c3c	2018-06-07 19:03:12	2018-06-21 15:34:32	2018-07-04 00:00:00	009c09f43998e	perfumaria	132.40	5a8608437f4b52bec5cd	14.05	5
17	116f0b0934	1	6087cfc70fd833cf2db63	2017-12-26 23:41:31	2018-01-08 22:36:36	2018-01-29 00:00:00	a47295965bd0	telefonica	27.99	0c00083494717bc3697f	15.10	5
18	85ce859df6	1	d0ff1a7468fca46b8f6c5f	2017-11-21 00:03:41	2017-11-27 18:28:00	2017-12-11 00:00:00	cce679660c66	cama_mesa_banho	17.90	70c498f6918af37d6444c	11.85	5
19	83018ec114	1	6340f96075fe9032e6c1	2017-10-26 15:54:26	2017-11-08 22:22:00	2017-11-23 00:00:00	c35498fbb435f	telefonica	76.00	219c5f9cf889bc85babbb	16.97	4
20	203096f03d	1	d69968853772c1a0a61	2017-09-18 14:31:30	2017-10-09 22:23:46	2017-09-28 00:00:00	5ac9d9e379c6f	beleza_saude	109.90	38cae21b1b57a959594c	8.96	2
21	f848643eec	1	68954feaf4dd638f3fbc	2018-03-15 08:52:40	2018-03-19 18:08:32	2018-03-29 00:00:00	2b46098948b	beleza_saude	79.99	f3d7d8d82a38f81f086b6	8.91	5
22	2807d0e504	1	b8df986511d928829c31	2018-02-03 20:37:35	2018-02-08 16:13:46	2018-02-21 00:00:00	6893767814d1	eletronicos	9.50	e905f4e4e372b175115c	7.78	3
23	95266dbfb7	1	451e48381edab7f1f6db	2018-01-08 07:55:29	2018-01-26 17:32:38	2018-02-21 00:00:00	bb7181410b4e	bebes	129.99	1d507207fdd8c75b7853f	57.58	4
24	f3e7c35915	1	9c9242ad7f1b52d926ea	2018-08-09 11:44:40	2018-08-13 18:24:27	2018-08-17 00:00:00	e99d69ef684c	papelaria	89.90	9a55cbb65c8fad47581d	14.21	5
25	fbf9ac61453	1	a25d5f94840d3c6a1a49	2018-02-20 23:46:53	2018-03-21 22:03:54	2018-03-12 00:00:00	7b717060aa78	cool_stuff	109.90	6a1a8e54de03ab98e6ef	15.53	2
26	acce194856	1	576ea0cab426cd8a00af	2018-06-04 00:00:13	2018-06-16 15:20:55	2018-07-18 00:00:00	d70f38e7f79c6	bebes	90.90	2d4303fd5e1eb4ab51c8	48.64	1
27	acce194856	2	576ea0cab426cd8a00af	2018-06-04 00:00:13	2018-06-16 15:20:55	2018-07-18 00:00:00	9451e630d725	bebes	39.50	2d4303fd5e1eb4ab51c8	48.64	1
28	dd78f560c2	1	f4a7ef6bd931f83d75d8	2018-03-12 01:50:26	2018-03-21 14:41:50	2018-03-28 00:00:00	00baba5b58e2	perfumaria	47.90	67904467df350a66376e	12.79	5
29	91b2a010e1	1	bd13608b9c6033892cef	2018-05-02 11:45:38	2018-05-16 20:56:24	2018-05-23 00:00:00	ba74c6b75d2a	relogios_presentes	178.99	8a4283cd714cfa8f3da9	13.69	4
30	eca9b0c993	1	1428917cd397d49fac0fc	2018-02-25 13:50:30	2018-03-27 23:29:14	2018-04-13 00:00:00	cd0b539123aa	ferramentas_jardim	30.10	0153550f6dd1a6bdc33	33.24	4
31	f70a0aff17d	1	1974875b4a1d2e2ee6df	2017-08-10 11:58:33	2017-08-18 14:28:02	2017-08-23 00:00:00	cafd558df4d3c	bebes	279.00	2f11691fc93df88fcaae	34.19	2
32	1790ea0b5	1	a6fecd9f434474cf8fcd8	2018-04-16 21:53:39	2018-05-05 12:28:34	2018-05-15 00:00:00	2d8f2be4f087e	climatizacao	186.90	764c587207b959994ab	38.00	1
33	989225ba6c	1	37363700139c1ae8f73b	2017-12-12 13:56:04	2018-01-03 18:03:36	2018-01-08 00:00:00	6b64362e8989	moveis_decoracao	49.00	3ee0f16b91a29ca331bb	14.10	4
34	d887b526c5	1	a2649503b92028291f01	2018-02-03 12:38:58	2018-02-02 00:07:55	2018-03-07 00:00:00	39a99428650d	brinquedos	84.90	57bec4c29a5113d8344	15.35	5
35	b276be48f0c	1	60cbe8a392b76916e84c2	2018-07-29 23:34:51	2018-07-31 22:48:50	2018-08-06 00:00:00	c6c1f263e076f	perfumaria	179.00	e71c2035f3e772a0a233f	9.41	5

2.4 基础查询验证

连接 Navicat，执行以下 SQL 语句验证数据正确入库：

查询 1：验证总行数

```
1 SELECT COUNT(*) AS total_rows FROM cleaned_flat_table;
```

The screenshot shows a SQL query execution interface. At the top, there are tabs for '保存' (Save), '查询创建工具' (Query Creation Tool), '美化 SQL' (Beautify SQL), '代码段' (Code Snippets), and '询问 AI' (Ask AI). Below these are dropdown menus for 'My_openGauss', 'ecommerce_db', and 'public'. To the right are buttons for '运行' (Run), '停止' (Stop), and '解释' (Explain). The query editor contains the SQL statement: `1 SELECT COUNT(*) AS total_rows FROM cleaned_flat_table;`. Below the editor, there are tabs for '消息' (Messages), '摘要' (Summary), and '结果 1' (Result 1). Under '结果 1', there are sub-tabs for '数据' (Data) and '信息' (Info). The '数据' tab is active, showing a table with one column 'total_rows' and one row with the value '110012'.

total_rows
110012

查询 2: 验证主键唯一性

```
1 SELECT order_id, order_item_id, review_id, COUNT(*)
2 FROM cleaned_flat_table
3 GROUP BY order_id, order_item_id, review_id
4 HAVING COUNT(*) > 1;
```

The screenshot shows a SQL query execution interface. The query editor contains the SQL statement: `1 SELECT order_id, order_item_id, review_id, COUNT(*)
2 FROM cleaned_flat_table
3 GROUP BY order_id, order_item_id, review_id
4 HAVING COUNT(*) > 1;`. Below the editor, there are tabs for '消息' (Messages), '摘要' (Summary), and '结果 1' (Result 1). Under '结果 1', there are sub-tabs for '数据' (Data) and '信息' (Info). The '数据' tab is active, showing a table with four columns: 'order_id', 'order_item_id', 'review_id', and 'count'. The first row of data shows '(N/A)' for all four columns.

order_id	order_item_id	review_id	count
(N/A)	(N/A)	(N/A)	(N/A)

查询 3: 验证约束字段范围

```
1 SELECT
2     MIN(price) AS min_price,
3     MAX(price) AS max_price,
4     MIN(freight_value) AS min_freight,
5     MAX(freight_value) AS max_freight,
6     MIN(review_score) AS min_score,
7     MAX(review_score) AS max_score
8 FROM cleaned_flat_table;
```

1	SELECT
2	MIN(price) AS min_price,
3	MAX(price) AS max_price,
4	MIN(freight_value) AS min_freight,
5	MAX(freight_value) AS max_freight,
6	MIN(review_score) AS min_score,
7	MAX(review_score) AS max_score
8	FROM cleaned_flat_table;

消息	摘要	结果 1
----	----	------

数据	信息
----	----

min_price	max_price	min_freight	max_freight	min_score	max_score
.85	6735.00	0.00	409.68	1	5

查询 4：抽样查看数据

```
1 SELECT
2     order_id,
3     order_item_id,
4     customer_unique_id,
5     product_category_name,
6     price,
7     freight_value,
8     review_score,
9     order_purchase_timestamp
10 FROM cleaned_flat_table
11 ORDER BY order_purchase_timestamp DESC
12 LIMIT 10;
```

1	SELECT
2	id,
3	order_id,
4	order_item_id,
5	customer_unique_id,
6	product_category_name,
7	price,
8	freight_value,
9	review_score,
10	order_purchase_timestamp
11	FROM cleaned_flat_table
12	ORDER BY order_purchase_timestamp DESC
13	LIMIT 10;

消息	摘要	结果 1
----	----	------

数据	信息
----	----

id	order_id	order_item_id	customer_unique_id	product_category_name	price	freight_value	review_score	order_purchase_time:
97357	35a972d7f8	1	24ac2b4327e25baf39f21	informatica_acessorios	84.99	8.76	5	2018-08-29 15:00:37
77033	03ef5dedbe	1	b701bebbdf478f550034	artigos_de_festas	24.90	8.33	3	2018-08-29 14:52:00
34884	168626408c	1	7febafa06d9d8f232a900	beleza_saude	45.90	15.39	5	2018-08-29 14:18:28
33296	0b223d92c2	1	5c58de6fb80e93396e2f	moveis_cozinha_area_de	209.00	46.48	5	2018-08-29 14:18:23
33297	0b223d92c2	2	5c58de6fb80e93396e2f	moveis_cozinha_area_de	209.00	46.48	5	2018-08-29 14:18:23
109937	5201848470	1	7a22d14aa3c359923850	brinquedos	63.90	9.20	1	2018-08-29 12:25:59
75005	d03ca98f59	1	0421e7a23f21e5d54efec	alimentos	109.90	9.52	5	2018-08-29 11:06:11
6333	d70442bc5e	1	21dbe8eabd00b34492a	esporte_lazer	6.90	7.39	5	2018-08-29 10:22:35
97707	912859fef5a	1	0c6d7218d5f3fa14514fd	relorios_presentes	169.80	8.45	5	2018-08-29 09:48:09
82626	fb39321145	1	b4dcade04bc548b7e3b	relorios_presentes	99.00	7.95	1	2018-08-29 09:14:11

3 Step3: 数据库规范化与数据可视化

3.1 函数依赖分析与规范化

3.1.1 宽表结构回顾

阶段二的 `cleaned_flat_table` 包含 13 列（含代理主键 `id`），业务候选键为 `(order_id, order_item_id, review_id)`。以下分析基于该候选键。

3.1.2 函数依赖集

编号	函数依赖	类型
FD1	<code>order_id</code> $\rightarrow$ <code>customer_unique_id</code> , <code>order_purchase_timestamp</code> , <code>order_delivered_customer_date</code> , <code>order_estimated_delivery_date</code>	部分依赖（候选键的真子集决定非主属性）
FD2	<code>(order_id, order_item_id)</code> $\rightarrow$ <code>product_id</code> , <code>price</code> , <code>freight_value</code>	部分依赖（候选键的真子集决定非主属性）
FD3	<code>product_id</code> $\rightarrow$ <code>product_category_name</code>	传递依赖（通过 <code>order_items</code> $\rightarrow$ <code>product_id</code> $\rightarrow$ <code>product_category_name</code> ）
FD4	<code>(review_id, order_id)</code> $\rightarrow$ <code>review_score</code>	部分依赖（候选键的真子集决定非主属性）

3.1.3 冗余与异常分析

宽表存在以下问题：

- 1. 数据冗余：
 - 同一客户的 `customer_unique_id` 在其所有订单明细行中重复出现
 - 同一订单的时间戳字段在该订单的每个明细×评价行中重复
 - 同一商品的 `product_category_name` 在每次出现时重复存储
- 2. 插入异常：无法单独添加一个新客户或新商品，必须伴随完整的订单明细和评价记录
- 3. 删除异常：删除某商品的最后一条订单明细，会导致该商品的类别信息丢失
- 4. 更新异常：修改某商品的类别名称，需要更新所有包含该商品的行，部分更新会导致数据不一致

3.2 概念设计

3.2.1 E-R 图

根据分析目标和数据特征，识别出 5 个实体及其联系：

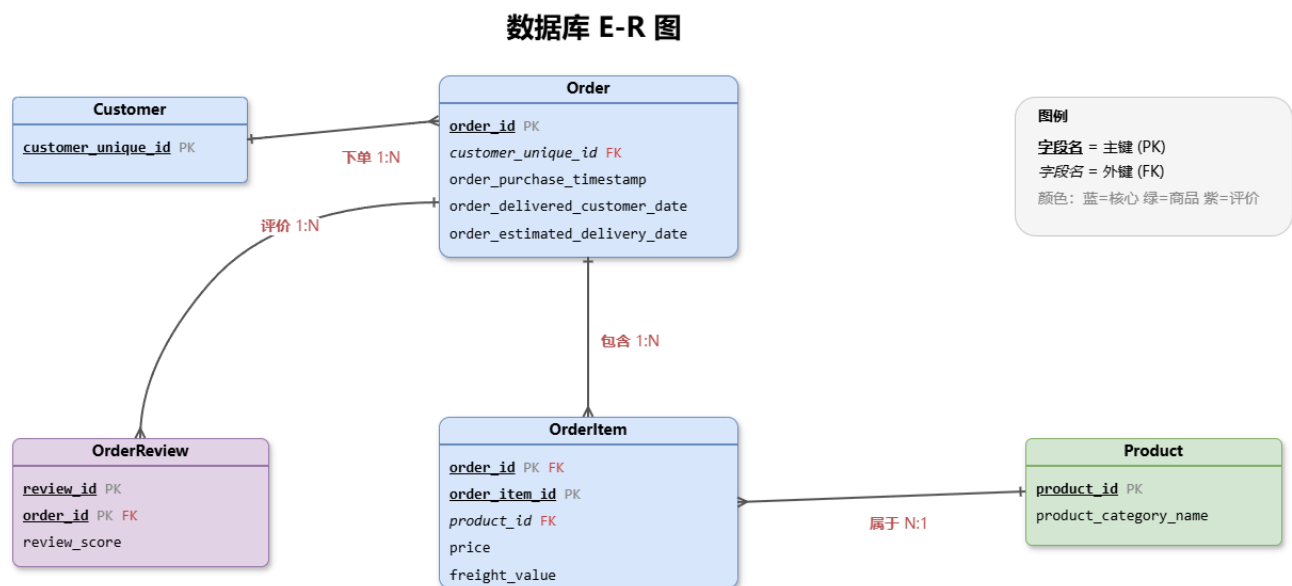
实体与属性：

实体	主键	其他属性
Customer	customer_unique_id	—
Order	order_id	order_purchase_timestamp, order_delivered_customer_date, order_estimated_delivery_date
OrderItem	(order_id, order_item_id)	price, freight_value
Product	product_id	product_category_name
OrderReview	(review_id, order_id)	review_score

联系与类型：

联系	类型	说明
Customer → Order	1:N	一个客户可下多个订单
Order → OrderItem	1:N	一个订单包含多个商品项
Product → OrderItem	1:N	一个商品可出现在多个订单项中
Order → OrderReview	1:N	一个订单可有多条评价

E-R图：



3.3 逻辑结构设计 with 规范化

3.3.1 E-R 图转关系模式

将 E-R 图中的 5 个实体直接转换为 5 个关系模式：

- **customers**(customer_unique_id)
- **orders**(order_id, *customer_unique_id*, order_purchase_timestamp, order_delivered_customer_date, order_estimated_delivery_date)
- **products**(product_id, product_category_name)
- **order_items**(order_id, order_item_id, *product_id*, price, freight_value)
- **order_reviews**(review_id, order_id, review_score)

其中下划线标注主键，斜体为外键。

3.3.2 3NF 分解过程

从宽表 R(order_id, order_item_id, review_id, customer_unique_id, order_purchase_timestamp, order_delivered_customer_date, order_estimated_delivery_date, product_id, product_category_name, price, freight_value, review_score) 出发：

第一步：消除部分依赖（达到 2NF）

- FD1: `order_id → customer_unique_id, 时间戳×3`，候选键的真子集决定非主属性 → 拆出 **orders** 表
- FD2: `(order_id, order_item_id) → product_id, price, freight_value`，候选键的真子集决定非主属性 → 拆出 **order_items** 表
- FD4: `(review_id, order_id) → review_score`，候选键的真子集决定非主属性 → 拆出 **order_reviews** 表

第二步：消除传递依赖（达到 3NF）

- FD3: 在 order_items 中，`product_id → product_category_name` 构成传递依赖（order_items 的主键 → product_id → product_category_name）→ 拆出 **products** 表
- 同时提取 **customers** 表作为 orders 外键的参照实体

分解结果：5 张表均满足 3NF，每个非主属性完全依赖于主键且不存在传递依赖。

3.3.3 参照完整性

外键	所在表	参照表
orders.customer_unique_id	orders	customers
order_items.order_id	order_items	orders
order_items.product_id	order_items	products
order_reviews.order_id	order_reviews	orders

3.4 架构迁移

3.4.1 3NF 建表脚本

对应 DDL 脚本为 `schema3.sql`，在 `proj1_3nf` schema 下创建 5 张规范化表：

```
1 CREATE SCHEMA IF NOT EXISTS proj1_3nf;
2
3 CREATE TABLE customers (
4     customer_unique_id VARCHAR(32) PRIMARY KEY
5 );
6
7 CREATE TABLE products (
8     product_id VARCHAR(32) PRIMARY KEY,
9     product_category_name VARCHAR(100) NOT NULL DEFAULT 'Unknown'
10 );
11
12 CREATE TABLE orders (
13     order_id VARCHAR(32) PRIMARY KEY,
14     customer_unique_id VARCHAR(32) NOT NULL,
15     order_purchase_timestamp TIMESTAMP NOT NULL,
16     order_delivered_customer_date TIMESTAMP NOT NULL,
```

```

17     order_estimated_delivery_date TIMESTAMP NOT NULL,
18     FOREIGN KEY (customer_unique_id) REFERENCES customers(customer_unique_id)
19 );
20
21 CREATE TABLE order_items (
22     order_id VARCHAR(32) NOT NULL,
23     order_item_id INTEGER NOT NULL CHECK (order_item_id > 0),
24     product_id VARCHAR(32) NOT NULL,
25     price NUMERIC(10,2) NOT NULL CHECK (price >= 0),
26     freight_value NUMERIC(10,2) NOT NULL CHECK (freight_value >= 0),
27     PRIMARY KEY (order_id, order_item_id),
28     FOREIGN KEY (order_id) REFERENCES orders(order_id),
29     FOREIGN KEY (product_id) REFERENCES products(product_id)
30 );
31
32 CREATE TABLE order_reviews (
33     review_id VARCHAR(32) NOT NULL,
34     order_id VARCHAR(32) NOT NULL,
35     review_score SMALLINT NOT NULL CHECK (review_score BETWEEN 1 AND 5),
36     PRIMARY KEY (review_id, order_id),
37     FOREIGN KEY (order_id) REFERENCES orders(order_id)
38 );

```

3.4.2 数据迁移

迁移脚本 `migrate_to_3nf.sql` 通过 `CREATE TEMP VIEW` 引用阶段二宽表，使用 `SELECT DISTINCT` 逐表提取数据：

目标表	迁移行数	说明
customers	92,753	DISTINCT customer_unique_id
products	32,070	DISTINCT product_id + category
orders	95,830	DISTINCT order_id + 时间戳
order_items	109,369	DISTINCT (order_id, order_item_id) + price/freight
order_reviews	96,359	DISTINCT (review_id, order_id) + score

3.4.3 迁移验证

执行 `validate_3nf.sql` 验证结果：

- 所有主键重复检查：0（无重复）
- 所有外键孤儿检查：0（无孤儿记录）
- 源宽表与目标表的粒度校验：全部匹配

```

1 DROP VIEW
2 CREATE VIEW
3     table_name | row_count
4 -----+-----
5 customers    |    92753
6 products     |    32070
7 orders       |    95830
8 order_items  |   109369
9 order_reviews |    96359
10 (5 rows)

```

```

11
12 duplicated_customer_keys
13 -----
14                                0
15 (1 row)
16
17 duplicated_product_keys
18 -----
19                                0
20 (1 row)
21
22 duplicated_order_keys
23 -----
24                                0
25 (1 row)
26
27 duplicated_order_item_keys
28 -----
29                                0
30 (1 row)
31
32 duplicated_review_keys
33 -----
34                                0
35 (1 row)
36
37 orphan_orders
38 -----
39                                0
40 (1 row)
41
42 orphan_order_items_by_order
43 -----
44                                0
45 (1 row)
46
47 orphan_order_items_by_product
48 -----
49                                0
50 (1 row)
51
52 orphan_reviews
53 -----
54                                0
55 (1 row)
56
57 expected_customers | actual_customers | expected_orders | actual_orders |
58 expected_products | actual_products | expected_order_items | actual_order_items |
59 -----+-----+-----+-----+
60 -----+-----+-----+-----+

```

```
61 |          92753 |          92753 |          95830 |          95830 |
62 |      32070 |      32070 |          96359 |          96359 |
63 |      109369 |      109369 |          96359 |          96359 |
    | (1 row)
```

3.5 数据可视化

使用 Streamlit 框架构建数据分析看板（`project1_app.py`），通过 `psycopg2` 连接 openGauss 数据库，实时从 3NF 表中查询数据。

3.5.1 核心指标概览

使用 `st.metric` 组件展示三个核心指标：

指标	SQL	数据来源
总订单数	COUNT(*) FROM orders	orders 表
总客户数	COUNT(*) FROM customers	customers 表
平均评分	AVG(review_score) FROM order_reviews	order_reviews 表

3.5.2 分析目标 1：物流时效与客户满意度

将订单按实际送达时间是否晚于预计送达时间分为"逾期送达"和"准时送达"两组，计算各组的平均评分，以柱状图展示对比。

3.5.3 分析目标 2：品类消费与复购行为差异

筛选家居家装类（`moveis_decoracao` 等 6 个类别）和美妆个护类（`beleza_saude`、`perfumaria`）客户，分别计算平均订单金额和复购率（下单 ≥ 2 次的客户占比），以并排柱状图展示。

3.5.4 分析目标 3：2017 年季度运费成本趋势

计算 2017 年各季度订单的平均运费占商品价格比例，以柱状图展示 Q1-Q4 的趋势变化。

3.5.5 原始数据抽样

通过 4 表 JOIN（orders + order_items + products + order_reviews）展示前 50 条规范化后的数据样本。

运行方式：

```
1 | streamlit run project1_app.py
```

3.5.6 结果展示

Olist Brazilian E-Commerce 数据分析看板

核心指标概览

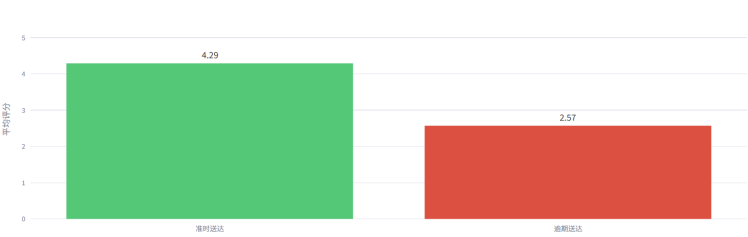
总订单数
95,830

总客户数
92,753

平均评分
4.16

分析目标 1：物流时效与客户满意度

2017-2018 年已送达订单：逾期送达 vs 准时送达的平均评分对比



评分基值
1.72 分

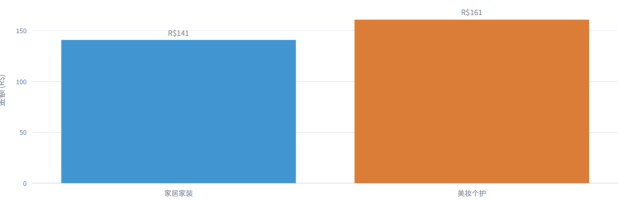
- 准时送达订单 87,902 单，平均评分 4.29
- 逾期送达订单 7,659 单，平均评分 2.57

逾期订单评分低 1.72 分，超过 1 分阈值，物流时效显著影响满意度。

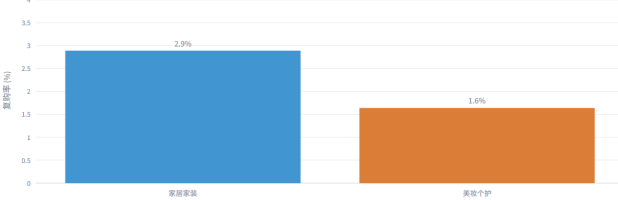
分析目标 2：品类消费与复购行为差异

2017-2018 年家居家装类 vs 美妆个护类客户的平均订单金额与复购率

平均订单金额 (R\$)



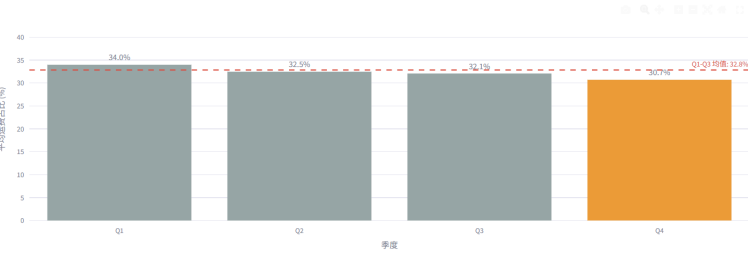
复购率 (%)



- 家居家装类客户 21,517 人，美妆个护类客户 11,409 人

分析目标 3：2017 年季度运费成本趋势

2017 年各季度订单的平均运费占商品价格比例，检验 Q4 是否显著高于前三季度



Q4 运费占比
30.7%

↓ 2.1% vs Q1-Q3 均值

Q4 运费占比未高于前三季度。

原始数据抽样

3NF 规范化后的数据样本 (4 表 JOIN, 前 50 行)

order_id	customer_unique_id	order_purchase_timestamp	order_item_id	product_category_name	price	freight_value	review_score	order_delivered_customer_date	order_estimated_delivery_date	
0	00010242fe8c5a6d1ba2dd792cb16214	871766c5855e863ffecc05988b23cb	2017-09-13 08:59:02	1	cool_stuff	58.9	13.29	5	2017-09-20 23:43:48	2017-09-29 00:00:00
1	00018777f2f0320c557190d7a144bd43	eb28e67c4c0b83846050ddfb8a35d051	2017-04-26 10:53:06	1	pet_shop	239.9	19.93	4	2017-05-12 16:04:24	2017-05-15 00:00:00
2	000229ec398224efca0657da4fc703e	3818d81c5709c39d6b2738a8d3a2474	2018-01-14 14:33:31	1	moveis_decoracao	199	17.87	5	2018-01-22 13:19:16	2018-02-05 00:00:00
3	00024acbcd0a6da1e931b0308114c75	a9861d436c098b2c2ddef00ba074622	2018-08-08 10:00:35	1	perfumaria	12.99	12.79	4	2018-08-14 13:32:39	2018-08-20 00:00:00
4	00042b26c5c9d7ce9d9abb4e52b4d9	64b576fb70d441e8f1b2d7d446e483c5	2017-02-04 13:57:51	1	ferramentas_jardim	199.9	18.14	5	2017-03-01 16:42:31	2017-03-17 00:00:00
5	00048cc3ae777c65dbb7d2a0634bdc1ea	85c835d128beae5b4ce8602c491bf385	2017-05-15 21:42:34	1	utilidades_domesticas	21.9	12.69	4	2017-05-22 13:44:35	2017-06-06 00:00:00
6	00054e8431b9d7675808bc819fb432	635d5ac1680f03288e72ada3a1035803	2017-12-10 11:53:48	1	telefonica	19.9	11.85	4	2017-12-18 22:03:38	2018-01-04 00:00:00
7	000576fe39319847cb9d288c5617fa6	fda4476abb6307ab3c415b7e6d026526	2018-07-04 12:08:27	1	ferramentas_jardim	810	70.75	5	2018-07-09 14:04:07	2018-07-25 00:00:00
8	0005a1a1728cd9d785b8e2b080904576c	639d23421f5517f69d0c3d6e6564c70e	2018-03-19 18:40:33	1	beleza_saude	145.95	11.65	1	2018-03-29 18:17:31	2018-03-29 00:00:00
9	0005f0442cb953dc1d21e1fb923495	0782c41380992a5a53348f9063d0eef6	2018-07-02 13:59:39	1	livros_tecnicos	53.99	11.4	4	2018-07-04 17:28:31	2018-07-23 00:00:00